

Reglerauslegung

1. Ziel der Aufgabe

Ziel dieser Aufgabe war die Auslegung eines PID-Reglers nach der Methode von **Chien, Hrones und Reswick (CHR)** für ein dynamisches System mit Totzeitapproximation sowie die Untersuchung des Systemverhaltens mit und ohne Regler.

Zusätzlich wurde ein interaktiver Schieberegler in Python implementiert, um die Reglerparameter K_p , T_i und T_d zu verändern und deren Einfluss auf die Sprungantwort zu analysieren.

2. Gegebenes System

Die Strecke ist gegeben durch:

$$G(s) = \frac{0.12}{4s^2 + 2s + 1}$$

mit

$$K = 0.04 \cdot KNR$$

und

$$KNR = 3$$

somit:

$$K = 0.12$$

Die Strecke ist ein **PT2-System**, wird für die CHR-Reglerauslegung jedoch als **PT1-System mit Totzeit (FOPDT)** angenähert.

Verwendete Approximation:

- Prozessverstärkung:

$$K_s = 0.12$$

- Zeitkonstante:

$$T = 2s$$

- Totzeit:

$$L = 0.5s$$

3. Chien-Hrones-Reswick Methode

Für einen PID-Regler ergeben sich nach CHR folgende Einstellregeln.

CHR für 0 % Überschwingen

$$K_p = 0.95 \cdot \frac{T}{K_s L}$$

$$T_i = 2.4L$$

$$T_d = 0.42L$$

Eingesetzt:

$$K_p = 31.67$$

$$T_i = 1.2$$

$$T_d = 0.21$$

CHR für 20 % Überschwingen

$$K_p = 1.2 \cdot \frac{T}{K_s L}$$

$$T_i = 2L$$

$$T_d = 0.42L$$

Ergebnis:

$$K_p = 40$$

$$T_i = 1$$

$$T_d = 0.21$$

4. PID-Regler

Verwendete Reglerübertragungsfunktion:

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

Geschlossener Regelkreis:

$$G_{cl}(s) = \frac{C(s)G(s)}{1+C(s)G(s)}$$

5. Implementierung in Python

Es wurde ein Python-Programm erstellt mit:

- Simulation der offenen Strecke (ohne Regler)
- Simulation des geschlossenen Regelkreises mit PID
- interaktive Slider für:
 - K_p
 - T_i
 - T_d
- Auswahl zwischen:
 - CHR 0 % Überschwingen
 - CHR 20 % Überschwingen

Bibliotheken:

- NumPy
 - Matplotlib
 - Python-Control
-

6. Vergleich mit und ohne Regler

Ohne Regler

Die offene Strecke besitzt stationär:

$$G(0) = 0.12$$

Das bedeutet:

- nur 12 % Sollwert-Erreichung
 - große bleibende Regelabweichung
 - langsames Verhalten
-

Mit PID-Regler

Mit PID-Regler:

- Sollwert wird erreicht
 - bleibende Regelabweichung verschwindet
 - schnelleres Einschwingen
 - Überschwingen über CHR-Einstellung beeinflussbar
-

7. Beobachtungen

Einfluss von K_p

- höheres K_p :
 - schneller
 - mehr Überschwingen
 - Instabilitätsrisiko
-

Einfluss von T_i

- kleines T_i :
 - aggressive Integralwirkung
 - schnellere Fehlerkorrektur
 - großes T_i :
 - ruhigeres Verhalten
 - langsamer
-

Einfluss von T_d

- reduziert Überschwingen
 - verbessert Dämpfung
 - stabilisiert das System
-

Einfluss der Totzeit

Die Totzeit verschlechtert:

- Stabilität
- Einschwingverhalten
- Regelgüte

und macht die Reglerauslegung anspruchsvoller.

8. Diagramm-Auswertung

Die Sprungantworten zeigen deutlich:

- offene Strecke erreicht Sollwert nicht
 - PID kompensiert stationären Fehler
 - CHR liefert brauchbare Anfangseinstellung
 - über Slider kann Optimierung durchgeführt werden
-

9. Fazit

Die CHR-Methode eignet sich gut zur ersten PID-Auslegung für Systeme mit Totzeit.

Durch den Vergleich mit und ohne Regler konnte gezeigt werden:

- Notwendigkeit eines Reglers
- Einfluss der PID-Parameter
- Verbesserung durch CHR-Abstimmung

Die Python-Simulation mit Schiebereglern ermöglicht zusätzlich ein anschauliches Verständnis des Regelkreisverhaltens.

Code ausführen

Benötigte Installation:

```
pip install numpy matplotlib control scipy
```

Programm starten:

```
python reglerauslegung.py
```

Anhang

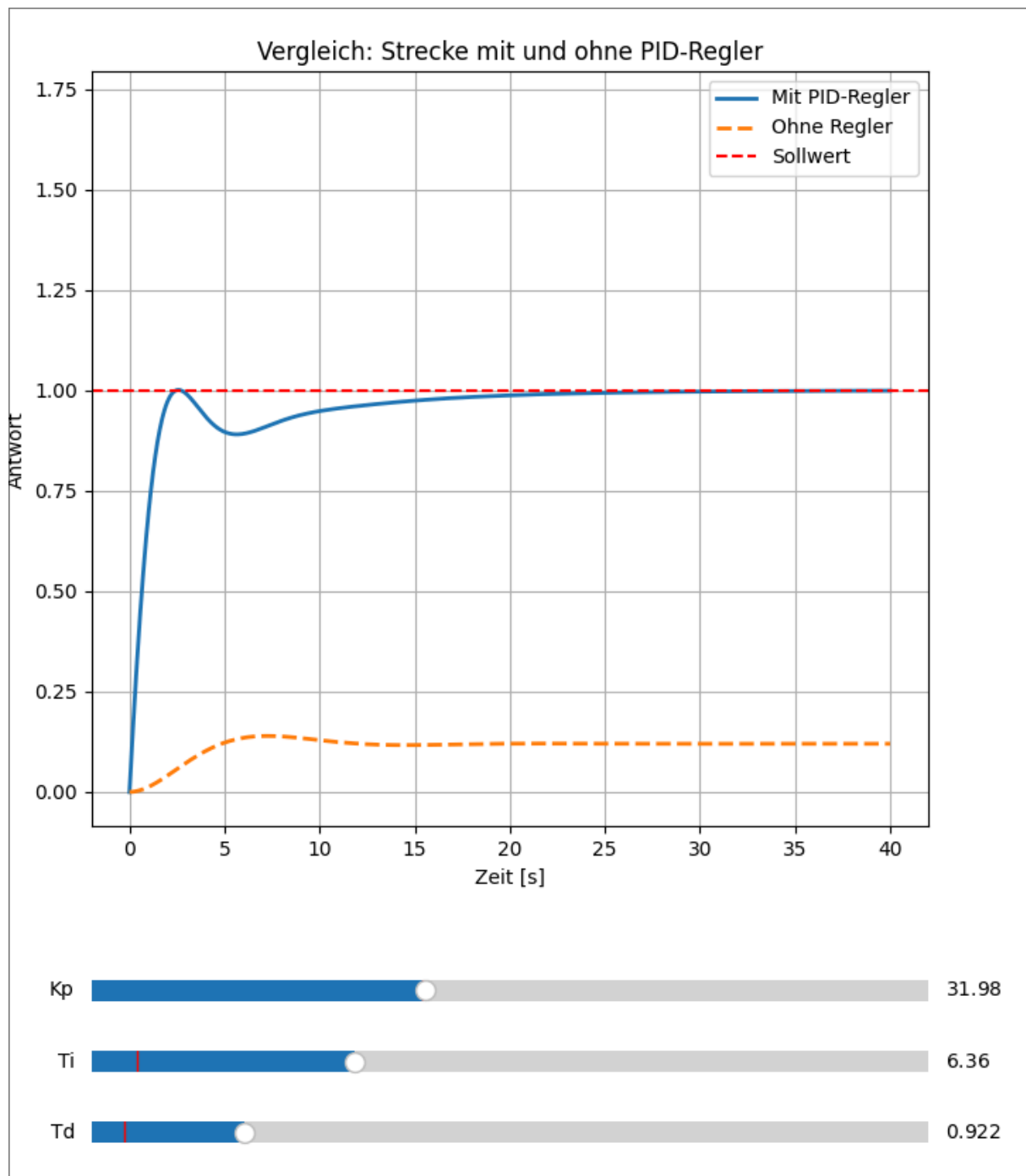
Verwendete Strecke:

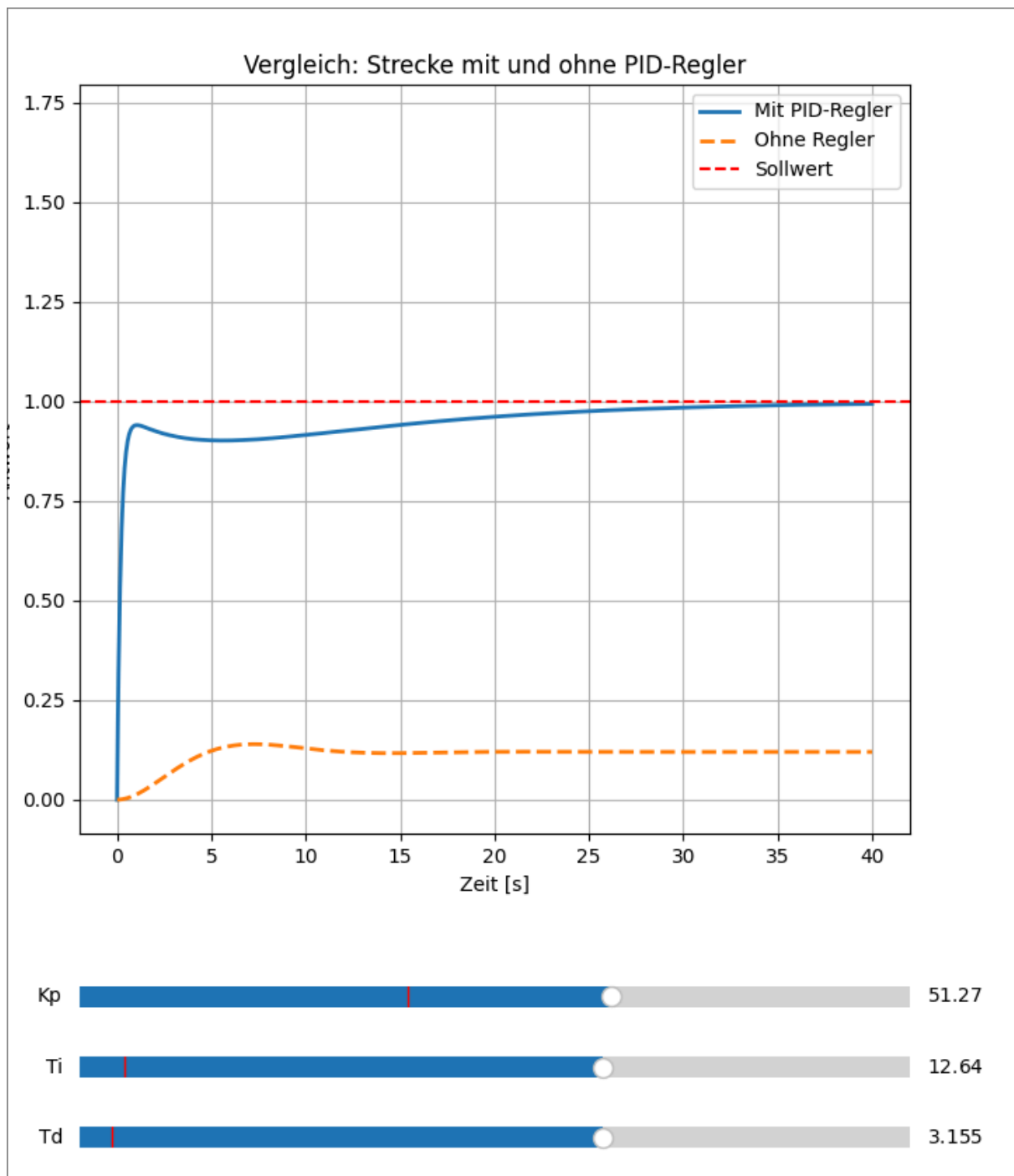
$$G(s) = \frac{0.12}{4s^2 + 2s + 1}$$

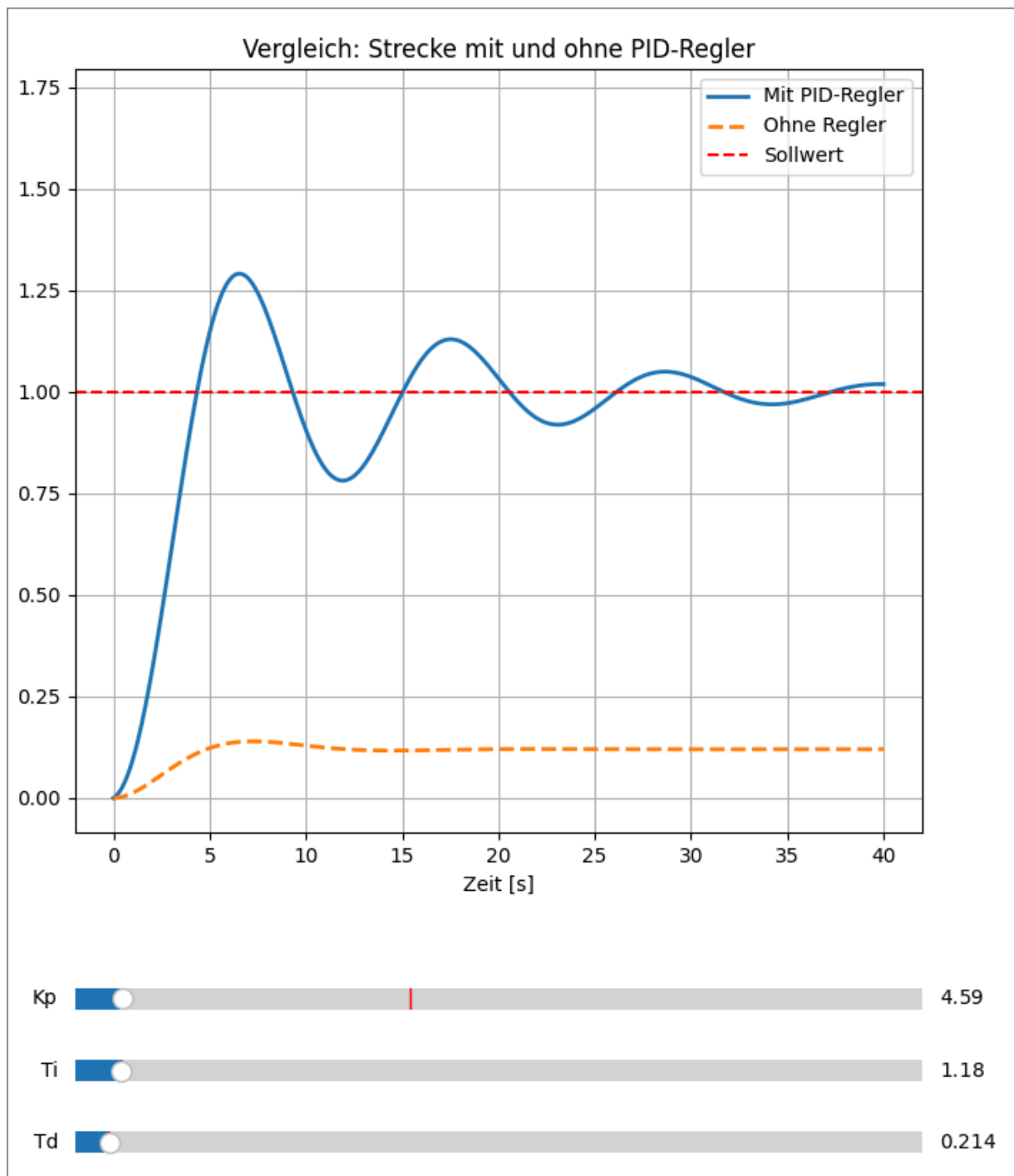
PID-Regler:

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

CHR-Formeln gemäß Chien-Hrones-Reswick.







CHR 0% Überschwingen

$K_p = 31.667$

$T_i = 1.2$

$T_d = 0.21$

CHR 20% Überschwingen

$K_p = 40.0$

$T_i = 1.0$

$T_d = 0.21$

code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider, RadioButtons
import control as ctl

# -----
# 1 Strecke
# KNR=3 -> K=0.04*3=0.12
#  $G(s)=0.12 / (4s^2+2s+1)$ 
# -----

K=0.12

G=ctl.TransferFunction(
    [K],
    [4,2,1]
)

# -----
# 2 PT1+Totzeit Approximation für CHR
# -----

Ks=0.12
T=2.0
L=0.5

# -----
# CHR Regeln
# -----
```

```

def chr_pid_0():
    Kp=0.95*T/(Ks*L)
    Ti=2.4*L
    Td=0.42*L
    return Kp,Ti,Td

def chr_pid_20():
    Kp=1.2*T/(Ks*L)
    Ti=2.0*L
    Td=0.42*L
    return Kp,Ti,Td

Kp0,Ti0,Td0=chr_pid_0()

# -----
# ---
# PID
# -----
# ---

def PID(Kp,Ti,Td):
    return ctl.TransferFunction(
        [Kp*Td*Ti,Kp*Ti,Kp],
        [Ti,0]
    )

def closed_loop(Kp,Ti,Td):
    C=PID(Kp,Ti,Td)
    return ctl.feedback(C*G)

# -----
# ---
# Simulation
# -----
# ---

t=np.linspace(0,40,1000)

```

```

# Ungeregelte Strecke
t_open,y_open=ctl.step_response(G,t)

# Geregelt
sys=closed_loop(Kp0,Ti0,Td0)
t_closed,y_closed=ctl.step_response(sys,t)

# -----
---
# Plot
# -----
---

fig,ax=plt.subplots(figsize=(11,7))
plt.subplots_adjust(left=0.25,bottom=0.35)

# MIT PID
[line_pid]=ax.plot(
    t_closed,
    y_closed,
    linewidth=2,
    label='Mit PID-Regler'
)

# OHNE PID
[line_open]=ax.plot(
    t_open,
    y_open,
    '--',
    linewidth=2,
    label='Ohne Regler'
)

ax.axhline(1,color='red',linestyle='--',label='Sollwert')

ax.set_title("Vergleich: Strecke mit und ohne PID-Regler")
ax.set_xlabel("Zeit [s]")
ax.set_ylabel("Antwort")

```

```

ax.grid()
ax.legend()

# -----
---
# Slider
# -----
---

axcolor='lightgoldenrodyellow'

ax_kp=plt.axes([0.25,0.22,0.65,0.03],facecolor=axcolor)
ax_ti=plt.axes([0.25,0.17,0.65,0.03],facecolor=axcolor)
ax_td=plt.axes([0.25,0.12,0.65,0.03],facecolor=axcolor)

sKp=Slider(ax_kp,'Kp',0.1,80,valinit=Kp0)
sTi=Slider(ax_ti,'Ti',0.1,20,valinit=Ti0)
sTd=Slider(ax_td,'Td',0.01,5,valinit=Td0)

# -----
---
# CHR Auswahl
# -----
---

rax=plt.axes([0.03,0.55,0.15,0.15],facecolor=axcolor)

radio=RadioButtons(
    rax,
    ('CHR 0%', 'CHR 20%')
)

# -----
---
# Update Funktion
# -----
---
```

```

def update(val):

    kp=sKp.val
    ti=sTi.val
    td=sTd.val

    sys=closed_loop(kp,ti,td)

    t,y=ctl.step_response(
        sys,
        np.linspace(0,40,1000)
    )

    line_pid.set_ydata(y)

    fig.canvas.draw_idle()

sKp.on_changed(update)
sTi.on_changed(update)
sTd.on_changed(update)

# -----
# Presets
# -----

def preset(label):

    if label=="CHR 0%":
        kp,ti,td=chr_pid_0()

    if label=="CHR 20%":
        kp,ti,td=chr_pid_20()

    sKp.set_val(kp)
    sTi.set_val(ti)
    sTd.set_val(td)

```

```

radio.on_clicked(preset)

plt.show()

# -----
# -----
# Werte ausgeben
# -----
# -----

print("CHR 0% Überspringen")
print("Kp =", round(Kp0,3))
print("Ti =", round(Ti0,3))
print("Td =", round(Td0,3))

kp20,ti20,td20=chr_pid_20()

print("\nCHR 20% Überspringen")
print("Kp =", round(kp20,3))
print("Ti =", round(ti20,3))
print("Td =", round(td20,3))

```